RHCSA Command Reference

Mihnea · RHCSA EX200 (RHEL 10) · RH124 · Last updated: 2026-10-17

Covers **RH124 chapters 1–6**. Add one section per chapter using the template at the end of this file.

Every chapter has the same three parts: **Commands** (what you type), **Key files** (where the config lives), and **Exam notes** (traps, how to verify, whether it survives a reboot).

> **On the exam you only have:** `man`, `info`, `cmd --help`, and `/usr/share/doc/`. This file is for building muscle memory. Practise finding everything in it offline too.

Chapter 1 — Get Started with Red Hat Enterprise Linux

Mostly concepts, very few commands.

Command	What it does	Example
`cat /etc/redhat-release`	Show the RHEL release you are running	`Red Hat Enterprise Linux release 10.x`
`cat /etc/os-release`	Distro name and version (works on Rocky/Alma too)	`grep VERSION /etc/os-release`
`uname -r`	Running kernel version	`uname -r`

Concepts to remember

- Open source: you can use, study, modify and redistribute the code.
- Upstream chain: **Fedora → CentOS Stream → RHEL**. Rocky and Alma are free RHEL rebuilds, fine for drills.
- A *distribution* = Linux kernel + userspace tools + package manager + installer, tested together.

Chapter 2 — Access the Command Line

Commands

Command	What it does	Example
`ssh user@host`	Log in to a remote system	`ssh student@serverb`
`ssh -i key user@host`	Log in with a specific private key	`ssh -i ~/lab.pem student@servera`
`exit` / `Ctrl+D`	Log out of the shell	`exit`
`whoami`	Which user am I?	`whoami`
`date`	Date and time, custom formats	`date +%R` (HH:MM), `date +%F` (YYYY-MM-DD)
`passwd`	Change **your own** password	`passwd`
`file`	Detect a file's type	`file /etc/passwd`
`cat`	Print one or more files	`cat file1 file2`
`less`	Page through a file	`less /etc/services`
`head -n N`	First N lines (default 10)	`head -n 5 /etc/passwd`
`tail -n N`	Last N lines (default 10)	`tail -n 3 /etc/passwd`
`tail -f`	Follow a file as it grows (Ctrl+C to stop)	`tail -f /var/log/messages`
`wc`	Count lines / words / bytes	`wc -l /etc/passwd`

`history`	Numbered list of past commands	`history | tail`
`!N` / `!string` / `!!`	Re-run command N / last one starting with *string* / last command	`!26`, `!ls`, `sudo !!`
`cmd1; cmd2`	Run commands one after another	`date; whoami`
`\` at end of line	Continue a long command on the next line	`head -n 3 \`, Enter, then `/etc/passwd`

Keyboard shortcuts

Keys	Action
`Tab` / `Tab Tab`	Complete a command, file name **or option** / list the possibilities
`Ctrl+A` / `Ctrl+E`	Jump to start / end of line
`Ctrl+U` / `Ctrl+K`	Delete from cursor to start / to end of line
`Ctrl+←` / `Ctrl+→`	Jump one word left / right
`Ctrl+R`	Search history backwards (press again for older matches)
`Esc .` or `Alt+.`	Insert the last argument of the previous command
`Ctrl+C` / `Ctrl+L`	Cancel the current command / clear the screen

Desktop (GNOME)

- `Super` key opens **Activities**; type `terminal` to open one.
- `Ctrl+Alt+F3` … `F6` switch to text virtual consoles; `Ctrl+Alt+F2` goes back to the graphical session.

Exam notes

> - `Tab Tab` also completes **long options**: `useradd --<Tab><Tab>` shows every option. Faster than opening the man page.
> - `ssh -i` refuses a private key that others can read: fix with `chmod 600 key`.
> - First SSH connection asks you to accept the host key. It is then stored in `~/.ssh/known_hosts`.

Chapter 3 — Manage Files from the Command Line

File system hierarchy (FHS)

Directory	Contents
`/`	Root of the whole tree
`/boot`	Kernel and boot loader files
`/dev`	Device files (`/dev/sda`, `/dev/null`)
`/etc`	System configuration files
`/home`	Regular users' home directories
`/root`	root's home directory
`/run`	Runtime data since last boot (wiped at reboot)
`/tmp`	Temporary files, world-writable (old files cleaned automatically)
`/usr`	Installed software: `/usr/bin`, `/usr/sbin`, `/usr/lib`, `/usr/share`
`/usr/local`	Locally built software
`/var`	Data that changes and persists across reboots: logs, caches, spools

`/bin`, `/sbin`, `/lib` are symlinks into `/usr`.

Navigate and list

Command	What it does	Example
`pwd`	Print current directory	`pwd`
`cd dir`	Change directory	`cd /etc`
`cd` / `cd ~`	Go to your home	`cd`
`cd -`	Go back to the previous directory	`cd -`
`cd ..`	Go up one level	`cd ../..`
`ls -l`	Long listing (perms, owner, size, date)	`ls -l /etc/passwd`
`ls -a`	Include hidden (dot) files	`ls -la ~`
`ls -R`	List recursively	`ls -R /etc/sysconfig`
`ls -d`	Show the directory itself, not its contents	`ls -ld /tmp`
`ls -i`	Show inode numbers	`ls -i file1 file2`
`ls -h`	Human-readable sizes (with `-l`)	`ls -lh`

Absolute path starts with `/` (`/home/student/file`). Relative path starts from where you are (`../docs/file`).

Create, copy, move, delete

Command	What it does	Example
`touch`	Create an empty file / update its timestamp	`touch notes.txt`
`mkdir`	Create a directory	`mkdir reports`
`mkdir -p`	Create a whole path, no error if it exists	`mkdir -p proj/2026/q4`
`cp src dst`	Copy a file	`cp /etc/hosts ~/hosts.bak`
`cp -r`	Copy a directory recursively	`cp -r dir1 dir2`
`cp -a`	Archive copy: keeps perms, owner, timestamps, SELinux context	`cp -a /etc/skel /tmp/skel`
`mv`	Move or rename	`mv old.txt new.txt`
`rm`	Delete a file	`rm file`
`rm -r`	Delete a directory and everything in it	`rm -r olddir`
`rm -i` / `rm -f`	Ask before each delete / never ask	`rm -ri dir`
`rmdir`	Delete an **empty** directory	`rmdir emptydir`

Links

Command	What it does	Example
`ln target link`	Hard link (another name for the same inode)	`ln file1 file1-hard`
`ln -s target link`	Symbolic (soft) link (a pointer to a path)	`ln -s /etc/hosts ~/hosts-link`
`ls -li`	Compare inode numbers and link counts	`ls -li file1 file1-hard`

	Hard link	Symbolic link
Points to	The inode (data)	A path name
Across file systems	No	Yes
To a directory	No	Yes
Original deleted	Data still reachable	Link breaks ("dangling")

Shell expansions

Pattern	Matches / expands to	Example
`*`	Any string, including empty	`ls *.conf`
`?`	Exactly one character	`ls file?.txt`
`[abc]`	One character from the set	`ls file[123]`
`[!abc]` or `[^abc]`	One character **not** in the set	`ls [!a]*`
`[[:alpha:]]` `[[:digit:]]` `[[:upper:]]` `[[:lower:]]` `[[:alnum:]]` `[[:space:]]` `[[:punct:]]`	Character classes	`ls [[:upper:]]*`
`{a,b,c}`	Brace list	`mkdir -p dir/{logs,data,tmp}`
`{1..5}` / `{a..e}`	Brace range	`touch file{1..5}.txt`
`~` / `~user`	Your home / user's home	`ls ~student`
`$VAR` / `${VAR}`	Variable value	`echo ${HOME}`
`$(cmd)`	Command substitution (output of cmd)	`echo "Today is $(date +%A)"`

Quoting	Effect	Example
`"..."`	Stops globbing and word splitting; `$VAR` and `$(...)` still expand	`echo "$HOME *"` → `/home/student *`
`'...'`	Stops **all** expansion	`echo '$HOME'` → `$HOME`
`\`	Escapes the next single character	`echo \$HOME` → `$HOME`

Exam notes

> - Use **absolute targets for symlinks** (`ln -s /data/file /opt/link`). A relative target is resolved from the link's directory, not from where you ran the command.
> - `cp -r dir1 dir2`: if `dir2` already exists you get `dir2/dir1`, not a copy into `dir2`. Check with `ls -R` afterwards.
> - Test a glob safely before deleting: run `ls pattern` first, then `rm pattern`.
> - Files and links are on disk, so they survive a reboot. Anything under `/run` or `/tmp` may not.

Chapter 4 — Get Help in Red Hat Enterprise Linux

Commands

Command	What it does	Example
`man cmd`	Open the manual page	`man useradd`
`man N topic`	Open a specific section	`man 5 passwd` (the file format, not the command)
`man -k word`	Search page names + descriptions (same as `apropos`)	`man -k password`
`man -f cmd`	Which sections have this page (same as `whatis`)	`man -f passwd`
`man -K word`	Search the **full text** of all pages (slow, asks per hit)	`man -K NOPASSWD`
`mandb`	Rebuild the index used by `man -k`	`sudo mandb`
`cmd --help`	Short usage summary *(allowed on the exam)*	`chage --help`
`info cmd`	GNU info docs *(allowed on the exam)*	`info coreutils`
`ls /usr/share/doc/`	Package docs and example configs *(allowed on the exam)*	`ls /usr/share/doc/sudo`

Man page sections

Section	Contains
1	User commands (`ls`, `passwd`)
2 / 3	System calls / library functions (C programming)
4	Special files (`/dev/*`)
5	File formats and config files (`passwd`, `fstab`, `sudoers`)
6	Games
7	Conventions and overviews (`regex`, `signal`)
8	System administration commands (`useradd`, `mount`)

Moving around inside `man` (it uses `less`)

Key	Action
`Space` / `b`	Page down / up
`/text`	Search forward; `n` next match, `N` previous
`g` / `G`	Jump to start / end
`q`	Quit

Exam notes

> - `man -k` says "*nothing appropriate*" → the index isn't built yet. Run `mandb` once, then retry.
> - Filter by section: `man -k user | grep '(8)'` lists only admin commands about users.
> - Most pages have an **EXAMPLES** section near the end: open the page and type `/EXAMPLE`.
> - Before using a config file format from memory, check `man 5 <file>`. For sudo: `man 5 sudoers`, then `/EXAMPLE`.

Chapter 5 — Create, View, and Edit Text Files

Redirection and pipes

Streams: **0** = stdin, **1** = stdout, **2** = stderr.

Syntax	What it does
`cmd > file`	stdout to file (**overwrites**)
`cmd >> file`	stdout to file (**appends**)
`cmd 2> file`	stderr to file
`cmd 2> /dev/null`	Throw errors away
`cmd &> file`	stdout **and** stderr to file (`&>>` appends)
`cmd > file 2>&1`	Same as `&>`, classic form (order matters)
`cmd < file`	Use file as stdin
`cmd1 | cmd2`	Pipe: stdout of cmd1 becomes stdin of cmd2
`cmd | less` / `| head` / `| wc -l`	Page / trim / count output
`cmd | tee file`	Show output **and** save it (`tee -a` appends)
`cmd | sudo tee /etc/file`	Write a root-owned file from a normal user

Vim

Keys	Action
`vim file`	Open (or create) a file
`i` / `a` / `o` / `O`	Insert before cursor / after cursor / new line below / new line above
`Esc`	Back to command (normal) mode
`:w` / `:q` / `:wq` (or `:x`) / `:q!`	Save / quit / save and quit / quit **without** saving
`u` / `Ctrl+R`	Undo / redo
`x`	Delete character
`dd` / `5dd`	Cut line / cut 5 lines
`yy` / `5yy`	Copy (yank) line / 5 lines
`p` / `P`	Paste after / before
`v` / `V` / `Ctrl+V`	Visual select: characters / lines / block
`gg` / `G` / `:15`	Go to first line / last line / line 15
`0` / `$`	Start / end of line
`/text` then `n` / `N`	Search forward, next / previous match
`:%s/old/new/g`	Replace every *old* with *new* in the whole file
`:set nu`	Show line numbers

`vimtutor`	30-minute interactive vim lesson (shell command)

Shell variables and environment

Command	What it does	Example
`VAR=value`	Set a shell variable (no spaces around `=`)	`COUNT=40`
`echo $VAR`	Read it	`echo $COUNT`
`export VAR`	Make it an environment variable (child processes see it)	`export EDITOR=vim`
`VAR=x cmd`	Set a variable for one command only	`LANG=fr_FR.UTF-8 date`
`unset VAR`	Remove the variable	`unset COUNT`
`export -n VAR`	Un-export, keep as shell variable	`export -n EDITOR`
`set | less` / `env`	All shell variables / only environment variables	`env | grep PATH`
`alias name='cmd'`	Create an alias; `alias` lists, `unalias` removes	`alias ll='ls -l'`
`which cmd` / `type cmd`	Where a command lives / is it an alias, builtin, or file	`type ll`
`source file` (or `. file`)	Re-read a config file in the current shell	`source ~/.bashrc`

Variable	Meaning
`PATH`	Directories searched for commands (`~/bin` and `~/.local/bin` are included by default on RHEL)
`HOME`, `USER`, `SHELL`, `PWD`	Home dir, user name, login shell, current dir
`PS1`	Prompt format
`HISTFILESIZE`, `HISTTIMEFORMAT`	History length, timestamps in `history` (`HISTTIMEFORMAT="%F %T "`)
`LANG`	Language / locale
`EDITOR`	Default editor used by other tools

Key files

File	Read when	Use for
`~/.bashrc`	Every interactive shell	Your aliases, functions, variables
`~/.bash_profile`	Login shells	Login-time settings (it also sources `~/.bashrc`)
`/etc/profile`, `/etc/bashrc`, `/etc/profile.d/*.sh`	Same, but for **all** users	System-wide settings

Exam notes

- Variables and aliases typed at the prompt **disappear at logout**. To make them persistent, put them in `~/.bashrc` (one user) or a file in `/etc/profile.d/` (everyone).
- `sudo echo text > /etc/file` fails: the `>` is done by **your** shell, not by sudo. Use `echo text | sudo tee /etc/file`.
- `cmd 2>&1 > file` is wrong: stderr still goes to the screen. Redirect stdout first: `cmd > file 2>&1`.
- `sort file > file` empties the file (the shell truncates it before `sort` reads it). Write to a new file.

Chapter 6 — Manage Local Users and Groups

Key files

File	Format
`/etc/passwd`	`name:x:UID:GID:GECOS(comment):home:shell`
`/etc/shadow`	`name:hash:lastchange:min:max:warn:inactive:expire:` (days; root-only)
`/etc/group`	`name:x:GID:member1,member2` (lists **supplementary** members only)
`/etc/gshadow`	Group passwords and admins
`/etc/login.defs`	Defaults for **new** users: `PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`, `UID_MIN`, `CREATE_HOME`
`/etc/default/useradd`	`useradd` defaults: shell, home base, skel (`useradd -D` shows them)
`/etc/skel/`	Files copied into every new home directory
`/etc/sudoers`	sudo rules (edit only with `visudo`)
`/etc/sudoers.d/`	Drop-in sudo rule files (preferred)

Hash field in `/etc/shadow`: `$6$...` = SHA-512, `$y$...` = yescrypt. A leading `!` = password locked or never set.

UID ranges: `0` root · `1–200` system (static, Red Hat) · `201–999` system (dynamic) · `1000+` regular users. GIDs follow the same scheme.

Primary group = the GID in `/etc/passwd` (new files get this group). By default each user gets a private group with the same name. **Supplementary groups** = extra memberships listed in `/etc/group`.

Who am I / who is that

Command	What it does	Example
`id` / `id user`	UID, GID and all groups	`id harry`
`groups user`	Group names only	`groups harry`
`getent passwd user`	The user's `/etc/passwd` line	`getent passwd harry`
`getent group grp`	The group's `/etc/group` line	`getent group wheel`

Become root

Command	What it does	Example
`su -`	Root **login** shell (root's environment; asks root's password)	`su -`
`su - user`	Login shell as another user	`su - harry`
`su - user -c 'cmd'`	Run one command as another user	`su - harry -c 'id'`
`sudo cmd`	Run one command as root (asks **your** password)	`sudo useradd bob`
`sudo -i`	Root login shell via sudo	`sudo -i`
`sudo -u user cmd`	Run as a user other than root	`sudo -u harry whoami`
`sudo -l`	What may **I** run with sudo?	`sudo -l`
`sudo -l -U user`	What may **user** run? (run as root)	`sudo -l -U harry`
`visudo`	Safely edit `/etc/sudoers` (syntax check on save)	`visudo`
`visudo -f file`	Safely edit a drop-in file	`visudo -f /etc/sudoers.d/harry`
`visudo -c`	Check syntax of sudoers + all drop-ins	`visudo -c`

sudoers rule syntax: `WHO HOSTS=(RUN_AS) COMMANDS`

Rule	Meaning
`%wheel ALL=(ALL) ALL`	Members of **wheel** can run anything (enabled by default)
`harry ALL=(ALL) NOPASSWD: ALL`	harry runs anything **without** a password
`%admins ALL=(ALL) ALL`	`%` = a group
`amy ALL=/usr/sbin/useradd, /usr/bin/passwd, !/usr/bin/passwd root`	Only these commands, and not changing root's password

Manage users

Command	What it does	Example
`useradd user`	Create a user (+ private group + home from `/etc/skel`)	`useradd harry`
`useradd -u UID`	Set a specific UID	`useradd -u 2001 sarah`
`useradd -g grp`	Set the **primary** group	`useradd -g devs tom`
`useradd -G g1,g2`	Set **supplementary** groups	`useradd -G wheel,devs ana`
`useradd -c "text"`	Comment / full name (GECOS)	`useradd -c "Ana Pop" ana`
`useradd -s shell`	Login shell	`useradd -s /sbin/nologin svc1`
`useradd -d dir -m`	Custom home directory, create it	`useradd -d /srv/web -m web`
`useradd -D`	Show default settings	`useradd -D`
`usermod -aG grp user`	**Add** a supplementary group (keeps the others)	`usermod -aG wheel harry`
`usermod -G grp user`	**Replace** all supplementary groups	`usermod -G devs harry`
`usermod -g grp user`	Change primary group	`usermod -g devs harry`
`usermod -c / -s / -u`	Change comment / shell / UID	`usermod -s /sbin/nologin harry`
`usermod -d dir -m user`	Move home directory	`usermod -d /home/h2 -m harry`

`usermod -l new old`	Rename the login	`usermod -l harold harry`
`usermod -L` / `-U`	Lock / unlock the password	`usermod -L harry`
`usermod -e DATE`	Account expiry date	`usermod -e 2026-12-31 harry`
`userdel user`	Delete user, **keep** home and mail	`userdel harry`
`userdel -r user`	Delete user **and** home + mail spool	`userdel -r harry`
`find / -nouser`	Find files whose owner no longer exists	`find / -nouser -o -nogroup`

Manage groups

Command	What it does	Example
`groupadd grp`	Create a group	`groupadd devs`
`groupadd -g GID grp`	Create with a specific GID	`groupadd -g 30000 sales`
`groupadd -r grp`	Create a system group (GID < 1000)	`groupadd -r appsvc`
`groupmod -n new old`	Rename a group	`groupmod -n engineers devs`
`groupmod -g GID grp`	Change GID	`groupmod -g 30001 sales`
`groupdel grp`	Delete a group (fails if it's someone's primary group)	`groupdel sales`
`gpasswd -d user grp`	Remove one user from one group	`gpasswd -d harry devs`

Passwords and aging

Command	What it does	Example
`passwd user`	Set a user's password (as root)	`passwd harry`
`passwd --stdin user`	Set the password from stdin, non-interactive (Red Hat option)	`echo 'Redhat123' | passwd --stdin harry`
`chpasswd`	Same idea, `user:password` pairs	`echo 'harry:Redhat123' | chpasswd`
`passwd -S user`	Short password status	`passwd -S harry`
`chage -l user`	Show all aging settings	`chage -l harry`
`chage -d 0 user`	Force password change at next login	`chage -d 0 harry`
`chage -M N user`	Max days a password is valid	`chage -M 90 harry`
`chage -m N user`	Min days between changes	`chage -m 7 harry`
`chage -W N user`	Warning days before expiry	`chage -W 14 harry`
`chage -I N user`	Inactive days after expiry before the account locks	`chage -I 5 harry`
`chage -E DATE user`	Account expiry date (`-E -1` removes it)	`chage -E 2026-12-31 harry`
`date -d "+N days" +%F`	Compute a date for `-E`	`chage -E $(date -d "+30 days" +%F) harry`

Exam notes

- **`usermod -G` without `-a` wipes the user's other supplementary groups.** Always `usermod -aG`. Verify with `id user`.
- **sudo task** ("harry runs anything as root without a password"): `echo 'harry ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/harry`, then `visudo -c`. Verify with `sudo -l -U harry`, or `su - harry -c 'sudo -n id'` (should print `uid=0(root)`).
- Files in `/etc/sudoers.d/` whose name contains a `.` or ends in `~` are **ignored**. Name it `harry`, not `harry.conf`.
- **`/etc/login.defs` only affects users created afterwards.** "All new users' passwords expire after 90 days" → `PASS_MAX_DAYS 90` there. Existing users → `chage -M 90 user`.
- "User must change password at first login" → `chage -d 0 user`. Check with `chage -l user` (shows *password must be changed*).
- `usermod -L` only locks the **password**; SSH key logins still work. To block everything, also expire the account: `usermod -L -e 1 user` (this is what `man usermod` recommends).
- `su -` needs root's password. If root has none (you skipped it at install), use `sudo -i`, or set one with `sudo passwd root`.
- "User has no interactive shell" → `-s /sbin/nologin`. Check the last field of `getent passwd user`.
- **RHEL 9 vs 10:** on RHEL 10 `passwd` is provided by the `shadow-utils` package instead of its own package. `--stdin` still works; `chpasswd` is the fallback.
- Everything in this chapter writes to files in `/etc`, so it **survives a reboot** automatically.